Advanced Web Technology

Lab

Experiment No.: 1

Create Registration page using HTML5 and CSS3

Experiment No. 1

1. **Aim:** Design a Registration Form using HTML5 and CSS3 on the selected problem statement**.**
2. **Objectives:** From this experiment, the student will be able to
   - To introduce programming languages and techniques associated with the World Wide Web.
3. **Outcomes:** The learner will be able to
   - Demonstrate the Frontend for Web applications.
4. **Software Required:** Web Browser, Text editor
5. **Theory:**

HTML is the standard markup language for Web pages. With HTML you can create your own Website. HTML5 is the latest and most enhanced version of HTML. The latest versions of Apple Safari, Google Chrome, Mozilla Firefox, and Opera all support many HTML5 features and Internet Explorer 9.0 will also have support for some HTML5 functionality.

HTML5 introduces a number of new elements and attributes that can help you in building modern websites. Here is a set of some of the most prominent features introduced in HTML5.

- New Semantic Elements − These are like <header>, <footer>, and <section>.
- Forms 2.0 − Improvements to HTML web forms where new attributes have been introduced for <input> tag.
- Persistent Local Storage − To achieve without resorting to third-party plugins.
- WebSocket − A next-generation bidirectional communication technology for web applications.
- Server-Sent Events − HTML5 introduces events which flow from web server to the web browsers and they are called Server-Sent Events (SSE).
- Canvas − This supports a two-dimensional drawing surface that you can program with JavaScript.
- Audio & Video − You can embed audio or video on your webpages without resorting to third-party plugins.

- Geolocation − Now visitors can choose to share their physical location with your web application.
- Microdata − This lets you create your own vocabularies beyond HTML5 and extend your web pages with custom semantics.
- Drag and drop − Drag and drop the items from one location to another location on the same webpage.

The markup for an HTML 5 document would look like the following −

```
<!DOCTYPE html>

<html>
  <head>
    <meta charset = "utf-8">
    <title>...</title>
  </head>
   <body>
    ...
    </body>
</html>
```

The different tags used in HTML are as follows:

Structural Tags

<nav>	Defines a section of navigation links.
<p>	Defines a paragraph.
<section>	Defines a section of a document, such as header, footer etc.
	Defines an inline styleless section in a document.
<summary>	Defines a summary for the <details> element.

Tag	Description
<a>	Defines a hyperlink.
<article>	Defines an article.

<aside>	Defines some content loosely related to the page content.
<body>	Defines the document's body.
 	Produces a single line break.
<details>	Represents a widget from which the user can obtain additional information or controls on-demand.
	Specifies a division or a section in a document.
<h1> to <h6>	Defines HTML headings.
<head>	Defines the head portion of the document that contains information about the document.
<header>	Represents the header of a document or a section.
<hgroup>	Defines a group of headings.
<hr>	Produce a horizontal line.
<html>	Defines the root of an HTML document.
<footer>	Represents the footer of a document or a section.

Metadata Tags

Tag	Description
<base>	Defines the base URL for all linked objects on a page.
<basefont>	Obsolete Specifies the base font for a page.
<link>	Defines the relationship between the current document and an external resource.
<meta>	Provides structured metadata about the document content.
<style>	Inserts style information (commonly CSS) into the head of a document.
<title>	Defines a title for the document.

Formatting Tags

Tag	Description
<abbr>	Defines an abbreviated form of a longer word or phrase.
<acronym>	Defines an acronym.
<address>	Specifies the author's contact information.
<b>	Displays text in a bold style.
<bdi>	Represents text that is isolated from its surrounding for the purposes of bidirectional text formatting.
<bdo>	Overrides the current text direction.
<big>	displays text in a large size.
<blockquote>	Defines a long quotation.
	Obsolete Align contents in the center of the enclosing block.
<cite>	Indicates a citation or reference to another source.
<code>	Specifies text as computer code.
<del>	Specifies a block of deleted text.
<dfn>	Specifies a definition.
	Specifies emphasized text.
<font>	Obsolete Defines font, color, and size for text.
<i>	Displays text in an italic style.
<ins>	Defines a block of text that has been inserted into a document.
<kbd>	Specifies text as keyboard input.
<mark>	Represents text highlighted for reference purposes.
<output>	Represents the result of a calculation.
<pre>	Defines a block of preformatted text.

<progress>	Represents the completion progress of a task.
<q>	Defines a short inline quotation.
<rp>	Provides fall-back parenthesis for browsers that that don't support ruby annotations.
<rt>	Defines the pronunciation of character presented in a ruby annotations.
<ruby>	Represents a ruby annotation.
<s>	Obsolete Displays text in strikethrough style.
<samp>	Specifies text as sample output from a computer program.
<small>	Displays text in a smaller size.
<strike>	Obsolete Displays text in strikethrough style.
	Indicate strongly emphasized text.
<sub>	Defines subscripted text.
<sup>	Defines superscripted text.
<tt>	Displays text in a teletype style.
<u>	Obsolete Displays text with an underline.
<var>	Defines a variable.
<wbr>	Represents a line break opportunity.

List Tags

Tag	Description
<dd>	Specifies a definition for a term in a definition list.
<dir>	Defines a directory list.
<dl>	Defines a definition list.
<dt>	Defines a term (an item) in a definition list.

	Defines a list item.
	Defines an ordered list.
<menu>	Represents a list of commands.
	Defines an unordered list.

Table Tags

Tag	Description
<caption>	Defines the title of a table.
<col>	Defines attribute values for one or more columns in a table.
<colgroup>	Specifies attributes for multiple columns in a table.
<table>	Defines a data table.
<tbody>	Groups a set of rows defining the main body of the table data.
<td>	Defines a cell in a table.
<tfoot>	Groups a set of rows summarizing the columns of the table.
<thead>	Groups a set of rows that describes the column labels of a table.
<th>	Defines a header cell in a table.
<tr>	Defines a row of cells in a table.

CSS3 modules :

CSS3 is collaboration of CSS2 specifications and new specifications, we can called this collaboration is **module**. Some of the modules are shown below

- Selectors
- Box Model
- Backgrounds
- Image Values and Replaced Content
- Text Effects
- 2D Transformations

- 3D Transformations
- Animations
- Multiple Column Layout
- User Interface

Syntax:

The CSS syntax is made up of three parts : a selector, a property and a value

selector {property: value}

The selector is normally the HTML element/tag you wish to define, the property is the attribute you wish to change, and each property can take a value. The property and value are separated by a colon, and surrounded by curly braces:

body {color: black}

If the value is multiple words, put quotes around the value:

p {font-family: "sans serif"}

If we wish to specify more than one property, we must separate each property with a semicolon. The example below shows how to define a center aligned paragraph, with a red text color:

p {text-align:center;color:red}

To make the style definitions more readable, you can describe one property on each line, like this: p{ text-align: center;

color: black;

font-family: arial }

Types Of CSS :

A. External Style Sheet :

- External CSS is a file that contains only CSS code and is saved with a .css file extension.

- With an external style sheet we can change the look of an entire web site by changing one file.
- Each page must link to the style sheet using the <link> tag.
- The <link> tag goes inside the head section.
- We can use one external style sheet with multiple HTML files.

<link rel="stylesheet" type="text/css" href="style.css">

B. Internal Style Sheet :

- Internal style sheet is defined inside the head section by using the <style> tag.
- An internal style sheet should be used when a single document has a unique style.

<style type="text/css"> hr{color:sienna} p{margin-left:20px}

body{background-image:url("images/back40.gif")}

</style>

C. Inline Style Sheet :

- Inline style sheet is also called as embedded style sheet.
- This is defined along with the HTML tags.
- Inline Style Sheet has more priority than Internal and External Style Sheet.

<p style="font-size:12px">

CS3 color:

CSS3 has Supported additional color properties as follows −

- RGBA colors
- HSLA color
- Opacity

RGBA stands for Red Green Blue Alpha. It is an extension of CSS2,Alpha specifies the opacity of a color and parameter number is a numerical between 0.0 to 1.0. A Sample syntax of RGBA as shown below −

#d1 {background-color: rgba(255, 0, 0, 0.5);}

HSL stands for hue, saturation, lightness.Here Huge is a degree on the color wheel, saturation and lightness are percentage values between 0 to 100%. A Sample syntax of HSL as shown below − #g1 {background-color: hsl(120, 100%, 50%);}

HSLA stands for hue, saturation, lightness and alpha. Alpha value specifies the opacity as shown RGBA. A Sample syntax of HSLA as shown below −

#g1 {background-color: hsla(120, 100%, 50%, 0.3);}

opacity is a thinner paints need black added to increase opacity. A sample syntax of opacity is as shown below −

#g1 {background-color:rgb(255,0,0);opacity:0.6;}

JavaScript is the world's most popular programming language. It is the language for HTML, for the web, for servers, PCs, laptops, tablets, phones, and more.

- JavaScripts in HTML must be inserted between <script> and </script> tags.
- JavaScripts can be put in the <body> and in the <head> section of an HTML page.

The <script> Tag

To insert a JavaScript into an HTML page, use the <script> tag. The <script> and </script> tells where the JavaScript starts and ends. The lines between the <script> and </script> contain the JavaScript:

<script>

alert("My First JavaScript");

</script>

JavaScript in <body>

Example

<!DOCTYPE html>

<html>

<body>.....

<script>

document.write("<h1>This is a heading</h1>"); document.write(" This is a paragraph ");

</script>

.....</body>

</html>

JavaScript Functions and Events

- The JavaScript statements, in the example above, are executed while the page loads.
- More often, we want to execute code when an **event** occurs, like when the user clicks a button.
- If we put JavaScript code inside a **function**, we can call that function when an event occurs.

JavaScript in <head> or <body>

- You can place an unlimited number of scripts in an HTML document.
- Scripts can be in the <body> or in the <head> section of HTML, and/or in both.
- It is a common practice to put functions in the <head> section, or at the bottom of the page. This way they are all in one place and do not interfere with page content.

External JavaScripts

Scripts can also be placed in external files. External files often contain code to be used by several different web pages. External JavaScript files have the file extension .js. To use an external script, point to the .js file in the "src" attribute of the <script> tag:

Example

<!DOCTYPE html>

<html>

<body><script src="myScript.js"></script></body>

</html>

JavaScript Form Validation

JavaScript can be used to validate data in HTML forms before sending off the content to a server. Form data that typically are checked by a JavaScript could be:

- has the user left required fields empty?
- has the user entered a valid e-mail address?
- has the user entered a valid date?
- has the user entered text in a numeric field?

Required Fields

The function below checks if a field has been left empty. If the field is blank, an alert box alerts a message, the function returns false, and the form will not be submitted:

function validateForm() {

```
var x = document.forms["myForm"]["fname"].value; if (x == null || x == "") {
alert("First name must be filled out"); return false;
}
}
```

The function above could be called when a form is submitted:

Example

```
<form name="myForm" action="demo_form.asp" onsubmit="return validateForm()" method="post">
First name:
<input type="text" name="fname">
<input type="submit" value="Submit">
</form>
```

6. Conclusion:

With the help of different HTML5 tags registration form is designed and styled using CSS3. The different input fields of form are validated using javascript.

7. Viva Questions:

- Difference between HTML and HTML5.
- List different tags in HTML5
- What is the need for validation?
- Where do you embed java script code in HTML documents?
- What are the types of CSS3

8. References:

- Ralph Moseley, M.T. Savaliya, “Developing Web Applications”, Wilely India, Second Edition, ISBN: 978-81-265-3867-6
- “Web Technology Black Book”, Dreamtech Press, First Edition, 978-7722-997
- https://www.tutorialspoint.com/html5/index.htm
- https://developer.mozilla.org/en-US/docs/Web/Guide/HTML/HTML5
- https://www.w3schools.com/html/html5_intro.asp

Advanced Web Technology

Lab

Experiment No.: 2

Password Strength Indicator and Star Rating System

Experiment No. 2

1. **Aim:** Write a program to create and build a password strength indicator and star rating system using jQuery

2. **Objectives:** From this experiment, the student will be able to
   - To introduce programming languages and techniques associated with the World Wide Web.
3. **Outcomes: The** learner will be able to
   - Demonstrate the Frontend for Web applications.

4. **Software Required :** Web Browser, Text editor

5. **Theory:**

JQuery is a lightweight, "write less, do more" and JavaScript library. The purpose of jQuery is to make it much easier to use JavaScript on your website. JQuery takes a lot of common tasks that require many lines of JavaScript code to accomplish, and wraps them into methods that you can call with a single line of code. JQuery also simplifies a lot of the complicated things from JavaScript, like AJAX calls and DOM manipulation.

The jQuery library contains the following features:

- HTML/DOM manipulation
- CSS manipulation
- HTML event methods
- Effects and animations
- AJAX
- Utilities

jQuery Syntax:

The jQuery syntax is tailor made for **selecting** HTML elements and performing some **action** on the element(s). With jQuery you select (query) HTML elements and perform "actions" on them. Basic syntax is: **$(*selector*).*action*()**

- A $ sign to define/access jQuery
- A (*selector*) to "query (or find)" HTML elements A jQuery *action*() to be performed on the element(s)

The Document Ready Event:

$(document).ready (function(){ *// jQuery methods go here...* });

This is to prevent any jQuery code from running before the document is finished loading (is ready).

It is good practice to wait for the document to be fully loaded and ready before working with it. This also allows you to have your JavaScript code before the body of your document, in the head jQuery selectors are one of the most important parts of the jQuery library. jQuery selectors

allow you to select and manipulate HTML element(s). jQuery selectors are used to "find" (or select) HTML elements based on their id, classes, types, attributes, values of attributes and much more. It's based on the existing CSS Selectors, and in addition, it has some own custom selectors.

All selectors in jQuery start with the dollar sign and parentheses: $().

The element Selector:

The jQuery element selector selects elements based on the element name. You can select all <p> elements on a page like this:

$("p")

Example

When a user clicks on a button, all <p> elements will be hidden: Example

```
$(document).ready(function(){ $("button").click(function(){ $("p").hide(); }); });
```

The #id Selector:

The jQuery #id selector uses the id attribute of an HTML tag to find the specific element.

An id should be unique within a page, so you should use the #id selector when you want to find a single, unique element.

To find an element with a specific id, write a hash character, followed by the id of the element: $("#test")

Example

When a user clicks on a button, the element with id="test" will be hidden:

Example

```
$(document).ready(function(){ $("button").click(function(){ $("#test").hide(); }); });
```

The .class Selector:

The jQuery class selector finds elements with a specific class.

To find elements with a specific class, write a period character, followed by the name of the class: $(".test")

Example

When a user clicks on a button, the elements with class="test" will be hidden:

Example

```
$(document).ready(function(){ $("button").click(function(){ $(".test").hide(); }); });
```

Functions In a Separate File:

If your website contains a lot of pages, and you want your jQuery functions to be easy to maintain, you can put your jQuery functions in a separate .js file. However,

sometimes it is preferable to place them in a separate file, like this (use the src attribute to refer to the .js file:

<head><script src="http://ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js"></script>

<script src="my_jquery_functions.js"></script></head>

Examples of jQuery Selectors Syntax	**Description**
$("*")	Selects all elements
$(this)	Selects the current HTML element
$("p.intro")	Selects all <p> elements with class="intro"
$("p:first")	Selects the first <p> element
$("ulli:first")	Selects the first element of the first
$("ulli:first-child")	Selects the first element of every
$("[href]")	Selects all elements with an href attribute
$("a[target='_blank']")	Selects all <a> elements with a target attribute value equal to "_blank"
$("a[target!='_blank']")	Selects all <a> elements with a target attribute value NOT equal to "_blank"
$(":button")	Selects all <button> elements and <input> elements of type="button"
$("tr:even")	Selects all even <tr> elements
$("tr:odd")	Selects all odd <tr> elements

The following table lists all the methods used to manipulate the HTML and CSS.

Method	**Description**
addClass()	Adds one or more class names to selected elements
after()	Inserts content after selected elements
append()	Inserts content at the end of selected elements
appendTo()	Inserts HTML elements at the end of selected elements

attr()	Sets or returns attributes/values of selected elements
before()	Inserts content before selected elements
clone()	Makes a copy of selected elements
css()	Sets or returns one or more style properties for selected elements
detach()	Removes selected elements (keeps data and events)
empty()	Removes all child nodes and content from selected elements
hasClass()	Checks if any of the selected elements have a specified class name
height()	Sets or returns the height of selected elements
html()	Sets or returns the content of selected elements
innerHeight()	Returns the height of an element (includes padding, but not border)
innerWidth()	Returns the width of an element (includes padding, but not border)
insertAfter()	Inserts HTML elements after selected elements
insertBefore()	Inserts HTML elements before selected elements
offset()	Sets or returns the offset coordinates for selected elements (relative to the document)
offsetParent()	Returns the first positioned parent element
outerHeight()	Returns the height of an element (includes padding and border)
outerWidth()	Returns the width of an element (includes padding and border)
position()	Returns the position (relative to the parent element) of an element
prepend()	Inserts content at the beginning of selected elements
prependTo()	Inserts HTML elements at the beginning of selected elements
prop()	Sets or returns properties/values of selected elements
remove()	Removes the selected elements (including data and events)
removeAttr()	Removes one or more attributes from selected elements
removeClass()	Removes one or more classes from selected elements
removeProp()	Removes a property set by the prop() method
replaceAll()	Replaces selected elements with new HTML elements

replaceWith()	Replaces selected elements with new content
scrollLeft()	Sets or returns the horizontal scrollbar position of selected elements
scrollTop()	Sets or returns the vertical scrollbar position of selected elements
text()	Sets or returns the text content of selected elements
toggleClass()	Toggles between adding/removing one or more classes from selected elements
unwrap()	Removes the parent element of the selected elements
val()	Sets or returns the value attribute of the selected elements (for form elements)
width()	Sets or returns the width of selected elements
wrap()	Wraps HTML element(s) around each selected element
wrapAll()	Wraps HTML element(s) around all selected elements
wrapInner()	Wraps HTML element(s) around the content of each selected element

6. Conclusion:

We studied and implemented syntax of JQuery, document ready event, syntax of element selector, syntax of id selector and syntax of class selector to design the web page.

7. Viva Questions:

- What is jQuery and how it is different from JavaScript?
- Is jQuery for client scripting or server scripting?
- Should we have multiple document.ready() function on the same page?
- Why do we use jQuery instead JavaScript?

8. References:

- Ralph Moseley, M.T. Savaliya, “Developing Web Applications”, Wilely India, Second Edition,
- ISBN: 978-81-265-3867-6
- “Web Technology Black Book”, Dreamtech Press, First Edition, 978-7722-997
- https://www.simplify.com/commerce/docs/tutorial/index
- https://api.jquery.com/category/event

Advanced Web Technology

Lab

Experiment No.: 3

Design a Homepage using React.js

Experiment No. 3

1. **Aim:** Design a Homepage using React.js
2. **Objectives:** From this experiment, the student will be able to
   - To use various front and back end Tools

   .
3. **Outcomes:** The learner will be able to
   - Create application using appropriate front end framework.

4. **Software Required:** Web Browser, Text editor

5. **Theory:**

ReactJS –
ReactJS, also known as React or React.js, is an open-source JavaScript library for frontend development. ReactJS is used to create an interactive user interface and it is popular among the developer community. Because of its simplicity, native approach, data binding, performance and testability, ReactJS is often chosen over its other competitive frameworks.

React Virtual DOM –
The most important part of web application development is DOM manipulation. Unfortunately, this process slow and time consuming in most JavaScript operations. The main reason for it is that the JavaScript frameworks try to update the DOM more than needed. For example, if the list of ten items is created and only one of them is updated, then the most JavaScript framework will rebuild the entire list. However, to solve this problem, React Virtual DOM was introduced. In ReactJS the Virtual DOM is an in-memory representation of a real DOM. Manipulating browser DOM objects is much slower than the React virtual DOM because nothing becomes drawn onscreen in the virtual DOM

React Components –
Components are very important in React. Components are small reusable User Interface elements, that provide data for the view. ReactJS enables one to create smaller components and combine them, nest them inside one another to form an entire User Interface. Components take input, also known as props and return react elements and these elements describe the User Interface contents. React components can also be written as a “function component”. They are similar to JavaScript functions.

6. Procedure:

1 - Create a react app in your desired location using the following command:

npx create-react-app app-name

A directory called app-name is created with the following default files:

```
├── README.md
├── node_modules
├── package.json
├── .gitignore
├── public
│   ├── favicon.ico
│   ├── index.html
│   ├── logo192.png
│   ├── logo512.png
│   ├── manifest.json
│   └── robots.txt
└── src
    ├── App.css
    ├── App.js
    ├── App.test.js
    ├── index.css
    ├── index.js
    ├── logo.svg
    └── serviceWorker.js`
```

2 - Install react-router-dom for routing requests by executing the following command:

npm install react-router-dom

3 - Wrap the App component.

There are two types of React routers, BrowserRouter (makes URLs like example.com/about) and HashRouter (makes URLs like example.com/#/about). We re using BrowserRouter in this example and use it to wrap the App component.

Your src/index.js file should include the following code:

```
import React from 'react'
import { render } from 'react-dom'
import { BrowserRouter } from 'react-router-dom'
import App from './App'
render(
  <BrowserRouter>
    <App />
  </BrowserRouter>,
  document.querySelector('#root')
)
```

4 - Create a file named src/pages/HomePage.js with the following code:

```
import React from "react";
export default function HomePage() {
return (
<>
<h1>Hey from HomePage</h1>
<p>This is your awesome HomePage subtitle</p>
</>
);
}
```

5 - Create a file named src/pages/UserPage.js with the following code:

```
import React from "react";
import { useParams } from "react-router-dom";
export default function UserPage() {
let { id } = useParams();
return (
<>
<h1>Hello there user {id}</h1>
<p>This is your awesome User Profile page</p>
</>
);
}
```

6 - Decide and incorporate the routers that you want to use using Switch and Route. Switch groups all routes together and ensures that they take the precedence from top-to-bottom. Route, on the other hand, defines individual routes.

Your App.js file should include the decided routes.

```
import React from 'react'
import { Route, Switch } from 'react-router-dom'
// We will create these two pages in a moment
import HomePage from './pages/HomePage'
import UserPage from './pages/UserPage'
export default function App() {
 return (
  <Switch>
   <Route exact path="/" component={HomePage} />
   <Route path="/:id" component={UserPage} />
  </Switch>
 )
}
```

The above code matches the root route (/) to HomePage and matches other pages dynamically to UserPage.

7 - Link to a page within the SPA using Link.
In the src/pages/HomePage.js file, include the following code:

```
import React from 'react'
import { Link } from 'react-router-dom'
export default function HomePage() {
 return (
  <div className="container">
   <h1>Home </h1>
   <p>
    <Link to="/your desired link">Your desired link.</Link>
   </p>
  </div>
 )
}
```

7. Conclusion:

The experiment aimed to design a homepage using React.js, focusing on the practical application of front-end and back-end tools. We have successfully utilized various tools and frameworks to construct a responsive and interactive homepage.

8. Viva Questions:

- What is React.js.
- Difference between a library like React.js and a framework like Angular
- What are the main advantages of using a component-based architecture in web development?
- Explain structure of your React.js project.

9. References:

- Ralph Moseley, M.T. Savaliya, “Developing Web Applications”, Wiley India, Second Edition, ISBN: 978-81-265-3867-6
- https://legacy.reactjs.org/docs/getting-started.html
- https://www.geeksforgeeks.org/react-tutorial/
- https://www.w3schools.com/REACT/react_intro.asp

Advanced Web Technology

Lab

Experiment No.: 4

Create a Simple Login Form using React.js

Experiment No. 4

1. **Aim:** Write a program to create a Simple Login Form using React.js

2. **Objectives:** From this experiment, the student will be able to

   - To use various front and back end Tools.

3. **Outcomes: The** learner will be able to
   - Create application using appropriate front end framework.

4. **Software Required :** Web Browser, Text editor

5. **Theory:**

   In ReactJS, creating a form can be a nightmare as it involves handling inputs and validating inputs. React-hook-form is a ReactJS library that simplifies the process of creating forms.

   The library provides all the features that a modern form needs. It is simple, fast, and offers isolated re-renders for elements.

   In this article, we will learn how to create a basic registration and login form in React. We will be using the React Hook Form library to complete this task. Let's look at the features and advantages of the React Hook Form library, and understand why is it used to create forms.

   React Hook Form Library
   React Hook Form is a popular third-party library that simplifies form management in React functional components using hooks. It provides a comprehensive set of functionalities to handle various aspects of form handling like state management, field handling, form submission, etc.

   Features of React Hook Form

   - Open-source
   - Supports TypeScript
   - DevTool for inspecting form data
   - Provides Form Builder – create forms by drag and drop
   - Supports for React-native

6. **Procedure:**
   Step 1: Create a React application by using the below commands

   npx create-react-app shopping-cart

   Step 2: Cd into the project folder

   cd shopping-cart

Step 3: Start the application using the below commands

npm start

or

yarn start

You will be redirected to your browser.

Step 4: Start Editing the UI and let's Create a Basic Registration Form. In the src/App.js file, delete all the code, and create your form.

App.js: Creating a simple react form containing name, email, and password fields.

Filename: App.js

App.css: Contains styles for the App.js component.

Filename: App.css

Step 5: Install the react-hook-form library

npm install react-hook-form

Step 6: Import useForm hook from react-hook-form. It will return your register, handlesubmit, and errors methods

register: This is used to handle input fields. We can access the input field later using the name given to it. In the above example, that is "firstname".

<input {...register("firstname")} />

handlesubmit: Is used to handle the form submission. It takes a custom method (eg: onSubmit). It will automatically collect field values.

```
const onSubmit = data =>  console.log(data);

<form onSubmit={handleSubmit(onSubmit)}>
    // input field 1
    // input field 2
    <input type="submit" />
</form>
```

errors: We use this to handle errors. if we leave "firstname" field empty it will set errors.first name = true

```
<input {...register("firstname", { required: true })} />
```

```
{errors.firstname && <span>This field is required</span>}
```

Let's use all concepts and create a form. Adding the register method to each input field and giving a name. Also handling form submission

Step 7: Storing values in localStorage:

As we are building a login form, we need to verify log-in credentials so we store the form data in local storage.

```
const onSubmit = (data) => {
   localStorage.setItem(data.email, JSON.stringify({
      name: data.name, password: data.password
   }));
   console.log(JSON.parse(localStorage.getItem(data.email)));
 };
```

localStorage provides **setItem** methods to store whatever we want in the form of key-value pairs of string. And **getItem** to fetch the stored data back with the help of a key. We use **JSON.stringify()** converts our object data into a string (setItem only takes values in the string) and **JSON.parse()** parses the string data into an object.

Building login Page: Create a new file called Login.jsx in the src folder and add the below code. We copied the code from the registration page and removed the name field from it.

7. Conclusion:

We built a basic React registration and login page. The project demonstrated the fundamental concepts of React, such as component-based architecture, state management using 'useState', and event handling in forms.

8. Viva Questions:

- What is React.js, and why is it suitable for creating a login form?
- What steps would you take to make this login form more secure?
-

9. References:

- Ralph Moseley, M.T. Savaliya, "Developing Web Applications", Wilely India, Second Edition,
- ISBN: 978-81-265-3867-6
- https://www.w3schools.com/howto/howto_css_login_form.asp
- https://www.geeksforgeeks.org/react-hook-form-create-basic-reactjs-registration

Advanced Web Technology

Lab

Experiment No.: 5

Connecting Your React.js Project with MongoDB

Experiment No. 5

1. **Aim:** Connecting your React.js Project with MongoDB
2. **Objectives:** From this experiment, the student will be able to
   - To use various front and back end Tools

   .
3. **Outcomes:** The learner will be able to
   - Create application using appropriate backend framework and databases.

4. **Software Required:** Web Browser, Text editor, MongoDB, ReactJs.

5. **Theory:**

In modern web development, combining a frontend framework like React.js with a NoSQL database like MongoDB allows for efficient data management and a responsive user interface. MongoDB is a document-oriented database that stores data in flexible, JSON-like documents, making it a perfect fit for applications that require scalable, real-time data. Connecting these technologies involves using a backend server (typically Node.js with Express) to handle API requests between the React frontend and MongoDB.

Key Concepts:

- **React.js**: A JavaScript library for building user interfaces, primarily for single-page applications.

- **MongoDB**: A NoSQL database that uses a document-based model to store data.

- **Express**: A web application framework for Node.js that simplifies building APIs.

- **Mongoose**: An ODM (Object Data Modeling) library for MongoDB and Node.js that provides a schema-based solution to model your application data.

6. Procedure

1. Set Up Your React Application
   1. Create a new React app using Create React App:
      bash
      npx create-react-app my-app
      cd my-app

2. Set Up the Backend
   1. Create a new directory for the backend:
      bash
      mkdir backend
      cd backend

2. Initialize a new Node.js project:
   bash
   npm init -y

3. Install necessary packages:
   bash
   npm install express mongoose cors dotenv

3. Create the Server
   1. Create a file named server.js:
   javascript
   const express = require('express');
   const mongoose = require('mongoose');
   const cors = require('cors');
   require('dotenv').config();

   const app = express();
   app.use(cors());
   app.use(express.json());

   const PORT = process.env.PORT || 5000;

   mongoose.connect(process.env.MONGO_URI, { useNewUrlParser: true,
   useUnifiedTopology: true })
   .then(() => console.log('MongoDB connected'))
   .catch(err => console.error(err));

   app.listen(PORT, () => {
   console.log(`Server running on port ${PORT}`);
   });

   2. Create a .env file in the backend folder with your MongoDB URI:

   MONGO_URI=your_mongodb_connection_string

4. Define Your Data Model
   1. Create a folder named models and create a file, e.g., Item.js:
   javascript
   const mongoose = require('mongoose');

   const ItemSchema = new mongoose.Schema({
   name: { type: String, required: true },
   quantity: { type: Number, required: true },
   });

   module.exports = mongoose.model('Item', ItemSchema);

5. Create API Routes
   1. Create a folder named routes and create a file, e.g., items.js:
   javascript
```
const express = require('express');
const router = express.Router();
const Item = require('../models/Item');

router.get('/', async (req, res) => {
    const items = await Item.find();
    res.json(items);
});

router.post('/', async (req, res) => {
    const newItem = new Item(req.body);
    const savedItem = await newItem.save();
    res.json(savedItem);
});

module.exports = router;
```

   2. Import and use the routes in server.js:
   javascript
```
const itemRoutes = require('./routes/items');
app.use('/api/items', itemRoutes);
```

6. Connect React with the Backend
   1. Install Axios in your React app:
   bash
```
npm install axios
```

   2. Fetch data from the backend in your React component:
   javascript
```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const App = () => {
    const [items, setItems] = useState([]);

    useEffect(() => {
        const fetchItems = async () => {
            const response = await axios.get('http://localhost:5000/api/items');
            setItems(response.data);
        };
        fetchItems();
    }, []);
```

```
    return (
      <div>
        <h1>Items</h1>
        <ul>
          {items.map(item => (
            <li key={item._id}>{item.name}: {item.quantity}</li>
          ))}
        </ul>
      </div>
    );
};

export default App;
```

7. Run Your Application
   1. Start the backend server:
      bash
      node server.js

   2. Start the React app in another terminal:
      bash
      npm start

   3. Access your application at http://localhost:3000.

7. Conclusion

Connecting a React.js project with MongoDB using Node.js and Express allows for a powerful full-stack development experience. By following these steps, you can set up a basic application that interacts with a MongoDB database, facilitating data management in your web application.

8. Viva Questions

1. What is the purpose of using Mongoose in a Node.js application?
2. Explain the structure of a document in MongoDB.
3. How does CORS work in the context of a React application?
4. What is the role of environment variables in your application setup?
5. How would you handle errors in your API routes?

9. References

1. https://reactjs.org/docs/getting-started.html
2. https://docs.mongodb.com/
3. https://expressjs.com/
4. https://mongoosejs.com/docs/guide.html

Advanced Web Technology

Lab

Experiment No.: 6

Design a Web Page using Nodejs

Experiment No. 6

1. **Aim:** Design a Web Page using Nodejs.

2. **Objectives:** From this experiment, the student will be able to

   - To use various front and back end Tools.

3. **Outcomes:** The learner will be able to
   - Explore the new feature of Node.js Webserver - Server and Clients related things.

4. **Software Required:** Web Browser, Text editor, Node JS

5. **Theory:**

Node.js is a powerful JavaScript runtime that enables developers to run JavaScript on the server side. Unlike traditional server-side environments, which often rely on frameworks, Node.js allows you to create web applications using only built-in modules. This provides full control over the server logic and enables you to build lightweight applications without additional dependencies. Designing a home page using only Node.js involves using the HTTP module to handle requests and serve HTML content directly.

Key Concepts:
- **Node.js**: A runtime that allows JavaScript to be executed server-side.
- **HTTP Module**: A built-in module in Node.js for creating HTTP servers and handling requests and responses.
- **Server Response**: The mechanism by which a server sends data back to the client (e.g., an HTML page).

6. **Procedure:**

1. Set Up Your Node.js Environment
   1. Create a new project directory:
      bash
      mkdir my-homepage
      cd my-homepage
   2. Initialize a new Node.js project (optional):
      bash
      npm init -y

2. Create the Server
   1. Create a file named server.js:
   javascript
   const http = require('http');

   const server = http.createServer((req, res) => {

```
res.statusCode = 200; // HTTP status code
res.setHeader('Content-Type', 'text/html'); // Set content type to HTML

// HTML content for the home page
const homePage = `
  <!DOCTYPE html>
  <html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home Page</title>
    <style>
      body {
        font-family: Arial, sans-serif;
        background-color: #f4f4f4;
        margin: 0;
        padding: 0;
        text-align: center;
      }
      header {
        background: #35424a;
        color: #ffffff;
        padding: 20px 0;
      }
      main {
        padding: 20px;
      }
      footer {
        background: #35424a;
        color: #ffffff;
        position: fixed;
        bottom: 0;
        width: 100%;
        padding: 10px 0;
      }
    </style>
  </head>
  <body>
    <header>
      <h1>Welcome to My Home Page</h1>
    </header>
    <main>
      <p>This is a simple home page designed using only Node.js.</p>
    </main>
    <footer>
      <p>&copy; 2024 My Home Page</p>
    </footer>
  </body>
  </html>
`;

res.end(homePage); // Send the HTML content as a response
```

```
});

const PORT = process.env.PORT || 3000;

server.listen(PORT, () => {
   console.log(`Server running at http://localhost:${PORT}`);
});
```

3. Run Your Application
    1. Start the server:
        bash
        node server.js
    2. Access your home page at http://localhost:3000 in your web browser.

7. Conclusion:

Designing a home page using only Node.js allows developers to create a simple yet functional web application without relying on additional frameworks. By using the HTTP module, you can directly handle requests and serve dynamic content. This method is particularly useful for lightweight applications or when learning the fundamentals of web server development.

8. Viva Questions

1. What is the purpose of the HTTP module in Node.js?
2. How does Node.js handle asynchronous requests?
3. What are the benefits of using Node.js for web development?
4. Explain how the server responds to a client request.
5. How would you modify the home page to include more dynamic content?

9. References

1. https://nodejs.org/en/docs/
2. https://developer.mozilla.org/en-US/docs/Web/HTTP
3. https://javascript.info/fetch
4. https://nodejs.dev/learneeks.org/react-hook-form-create-basic-reactjs-registration

Advanced Web Technology Lab

Experiment No.: 7

Designing a Simple Form using Express.js in Node.js.

Experiment No. 7

1. **Aim:** Designing a Simple Form using Express.js in Node.js
2. **Objectives:** By the end of this process, users will be able to:

   Understand how to register a domain.

   Deploy a website using VS Code.

   Manage DNS settings and link the domain with the hosting server

3. **Outcomes:** Users will learn the essential steps for hosting a website, from domain registration to deployment and management using VS Code.

4. **Software Required:** Node.js (Runtime environment)
   Express.js (Web framework)
   Postman or a web browser (to test the form submission)
   Code editor (e.g., VS Code)
   npm (Node package manager, usually comes with Node.js).

5. **Theory:**

   Express.js -
   It is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. Express.js simplifies the creation of server-side applications and allows developers to create routes to handle different HTTP requests.
   In this experiment, we will design a simple HTML form that takes user input, submits it to the server, and processes the form data using Express.js. The form submission will be handled using the POST method, which sends data to the server for further processing. We will use body-parser middleware to parse incoming request bodies in a middleware before handlers.

6. **Procedure:**

Step 1: Install Node.js and Initialize the Project

1. **Install Node.js** if you haven't already from the official website.
2. Open a terminal/command prompt and navigate to your project directory.
3. Run npm init -y to initialize the project and create a package.json file.

Step 2: Install Express.js

1. In the terminal, run the following command to install Express.js:

```
npm install express
```

npm install express

Step 3: Set Up the Express Server

1. Create a file named server.js in the project directory.
2. Inside server.js, set up a basic Express server

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();

app.use(bodyParser.urlencoded({ extended: true })); // Middleware to parse form data

// Serve the HTML form
app.get('/', (req, res) => {
  res.send(`
    <form action="/submit" method="POST">
      <label for="name">Name:</label>
      <input type="text" name="name" id="name" required />
      <label for="email">Email:</label>
      <input type="email" name="email" id="email" required />
      <button type="submit">Submit</button>
    </form>
  `);
});

// Handle form submission
app.post('/submit', (req, res) => {
  const { name, email } = req.body;
  res.send(`Form submitted successfully! Name: ${name}, Email: ${email}`);
});

// Start the server
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Step 4: Run the Server

1. Run the following command in the terminal to start the server:

```
node server.js
```

Step 5: Test the Form

1. Open a browser and navigate to http://localhost:3000.
2. Fill in the form fields with your name and email, and click **Submit**.
3. You should see a response displaying the submitted data.

7. Result:

- A simple form was created using HTML and served via an Express.js server.
- The form data was successfully submitted using the POST method and processed on the server.
- The submitted data (name and email) was displayed on the webpage as a response.

8. Conclusion:

In this experiment, we demonstrated how to set up a Node.js server using Express.js to create a form and handle form submissions. The form data was parsed and processed using the POST method, showcasing how Express.js can handle HTTP requests in a web application. This experiment highlights the ease and flexibility of using Express.js for backend development.

9. Viva Questions:

- What is React.js.
- Define what are Git Repository.
- What are the main advantages of using HashRouter?
- Explain structure of your React.js project.

10. References:

- Ralph Moseley, M.T. Savaliya, “Developing Web Applications”, Wilely India, Second Edition, ISBN: 978-81-265-3867-6
- https://legacy.reactjs.org/docs/getting-started.html
- https://www.geeksforgeeks.org/react-tutorial/
- https://www.w3schools.com/REACT/react_intro.asp

Advanced Web Technology

Lab

Experiment No.: 8

Implementing Payment Gateway Integration in a Web Application

Experiment No. 8

1. **Aim:** Program for Payment Gateway Integration in a Web Application
2. **Objectives:** By the end of this process, users will be able to:

   Understand how to register a domain.

   Deploy a website using VS Code.

   Manage DNS settings and link the domain with the hosting server

3. **Outcomes:** Users will learn the essential steps for hosting a website, from domain registration to deployment and management using VS Code.

4. **Software Required:** Node.js (Runtime environment)
   Express.js (Web framework)
   Postman or a web browser (to test the form submission)
   Code editor (e.g., VS Code)
   npm (Node package manager, usually comes with Node.js).

5. **Theory:**

Express.js -
It is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. Express.js simplifies the creation of server-side applications and allows developers to create routes to handle different HTTP requests. In this experiment, we will design a simple HTML form that takes user input, submits it to the server, and processes the form data using Express.js. The form submission will be handled using the POST method, which sends data to the server for further processing. We will use body-parser middleware to parse incoming request bodies in a middleware before handlers.

A **payment gateway** is a service used by merchants to process payments, providing a secure connection between the buyer, the merchant, and the financial institution involved. In modern web applications, payment gateways like Stripe, Razorpay, and PayPal offer APIs that allow seamless integration of payment functionality.

In this experiment, we will integrate a payment gateway (e.g., Stripe) into a web application using **Express.js**. The client will make a payment request, which will be sent to the payment gateway's API. Upon successful payment, a confirmation will be shown to the user.

6. Procedure:

Step 1: Install Node.js and Initialize the Project

1. **Install Node.js** if you haven't already from the official website.
2. Open a terminal/command prompt and navigate to your project directory.
3. Run npm init -y to initialize the project and create a package.json file.

Step 2: Install Required Packages

1. Install Express and a payment gateway library (e.g., Stripe) by running

```
npm install express stripe
```

Step 3: Set Up the Payment Gateway Account

1. Sign up for a **Stripe account** (or your chosen gateway) at Stripe.
2. Get your **API keys** from the Stripe dashboard (publishable key and secret key).

Step 4: Create the Server and Payment Route

1. Create a file named server.js and set up a basic Express server with payment integration

javascript Copy code

```javascript
const express = require('express');
const stripe = require('stripe')('your_secret_key_here'); // Replace with your Stripe
const bodyParser = require('body-parser');
const app = express();

app.use(bodyParser.json());

// Serve the payment form
app.get('/', (req, res) => {
  res.send(`
    <form action="/pay" method="POST">
      <script
```

Copy code

```
      <script
        src="https://checkout.stripe.com/checkout.js" class="stripe-button"
        data-key="your_publishable_key_here"  // Replace with your Stripe Publishable
        data-amount="5000"
        data-name="Test Payment"
        data-description="Test Payment Gateway Integration"
        data-currency="usd">
      </script>
    </form>
  `);
});

// Handle the payment
app.post('/pay', async (req, res) => {
  try {
```

```
// Handle the payment
app.post('/pay', async (req, res) => {
  try {
    const { token } = req.body; // Token from the frontend
    const charge = await stripe.charges.create({
      amount: 5000, // Amount in cents (5000 = $50.00)
      currency: 'usd',
      source: token,
      description: 'Test Payment',
    });
    res.send('Payment Successful');
  } catch (error) {
    res.send('Payment Failed: ' + error.message);
  }
});
```

```
// Start the server
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Step 5: Test the Payment Gateway Integration

1. Run the server using

```
node server.js
```

 Open a browser and navigate to http://localhost:3000.

 The payment form will be displayed. You can fill in the test card details provided by Stripe (e.g., card number 4242 4242 4242 4242, expiration 12/34, and any CVV).

 After submitting the form, you should see a confirmation of successful payment.

7. Result:

- Successfully integrated a payment gateway (e.g., Stripe) into the web application.
- The user can submit payments through the form, and the backend processes the payment through the gateway's API.

- The result of the payment (success or failure) is displayed to the user.

8. Conclusion:

In this experiment, we successfully implemented a basic payment gateway integration using Stripe in an Express.js web application. The integration demonstrated how the frontend interacts with a payment gateway to collect payment information securely, and how the backend processes the transaction using the payment gateway's API. This experiment is a fundamental step in building e-commerce platforms and applications requiring payment processing.development.

9. Viva Questions:

- What is React.js.
- Define what are Git Repository.
- What are the main advantages of using HashRouter?
- Explain structure of your React.js project.

10. References:

- Ralph Moseley, M.T. Savaliya, “Developing Web Applications”, Wilely India, Second Edition, ISBN: 978-81-265-3867-6
- https://legacy.reactjs.org/docs/getting-started.html
- https://www.geeksforgeeks.org/react-tutorial/
- https://www.w3schools.com/REACT/react_intro.asp

Advanced Web Technology

Lab

Experiment No.: 9

Hosting the Website with Domain Registration.

Experiment No. 9

1. **Aim:** Hosting the Website with Domain Registration Process for VS Code
2. **Objectives:** By the end of this process, users will be able to:

   Understand how to register a domain.

   Deploy a website using VS Code.

   Manage DNS settings and link the domain with the hosting server

3. **Outcomes:** Users will learn the essential steps for hosting a website, from domain registration to deployment and management using VS Code.
4. **Software Required:** Web Browser, GitHub, VS Code.
5. **Theory:**

ReactJS –
ReactJS, also known as React or React.js, is an open-source JavaScript library for frontend development. ReactJS is used to create an interactive user interface and it is popular among the developer community. Because of its simplicity, native approach, data binding, performance and testability, ReactJS is often chosen over its other competitive frameworks.

React Virtual DOM –
The most important part of web application development is DOM manipulation. Unfortunately, this process slow and time consuming in most JavaScript operations. The main reason for it is that the JavaScript frameworks try to update the DOM more than needed. For example, if the list of ten items is created and only one of them is updated, then the most JavaScript framework will rebuild the entire list. However, to solve this problem, React Virtual DOM was introduced. In ReactJS the Virtual DOM is an in-memory representation of a real DOM. Manipulating browser DOM objects is much slower than the React virtual DOM because nothing becomes drawn onscreen in the virtual DOM

React Components –
Components are very important in React. Components are small reusable User Interface elements, that provide data for the view. ReactJS enables one to create smaller components and combine them, nest them inside one another to form an entire User Interface. Components take input, also known as props and return react elements and these elements describe the User Interface contents. React components can also be written as a “function component”. They are similar to JavaScript

functions.

GitHub -
GitHub is a developer platform that allows developers to create, store, manage and share their code. It uses Git software, providing the distributed version control of Git plus access control, bug tracking, software feature requests, task management, continuous integration

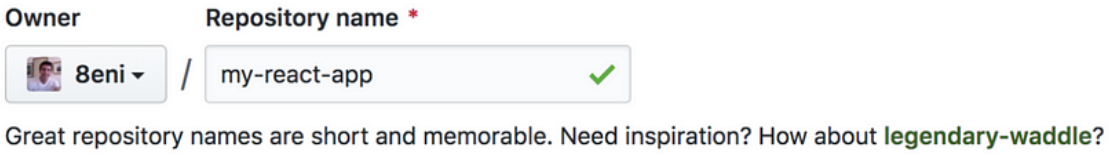
Projects on GitHub can be accessed and managed using the standard Git command-line interface; all standard Git commands work with it. GitHub also allows users to browse public repositories on the site. Multiple desktop clients and Git plugins are also available. In addition, the site provides social networking-like functions such as feeds, followers, and a social network graph to display how developers work on their versions of a repository is newest.

Anyone can browse and download public repositories, but only registered users can contribute content to repositories. With a registered user account, users can have discussions, manage repositories, submit contributions to others' repositories, and review changes to code.

6. Procedure:

Create GitHub repository and add React application

Navigate to your GitHub account and create a new repository. Give name as my react-app.



Clone our repository locally and run create-react-app from the same directory you cloned too.

$: git clone https://github.com/8eni/my-react-app.git
$: npx create-react-app my-react-app

Add Routing to our application

For routing, we will use the react-router-dom package. Navigate to our new directory and install the package

$: cd my-react-app
$: npm install react-router-dom

Then import HashRouter, Route, and Link from react-router-dom into our application. It is important to note we are using HashRouter here as without it on browser refresh, we would get a 404 on our GitHub pages live URL.

```
// App.js

import React, { Component } from 'react';

import { HashRouter, Route, Link } from "react-router-dom";
```

Next replace the render() method in App.js with the following.

```
// App.js

render() {
 return (
  <HashRouter basename='/'>
   <div>
    <ul>
     <li><Link to="/">Home</Link></li>
     <li><Link to="/about">About</Link></li>
    </ul>
    <hr />
    <Route exact path="/" component={Home} />
    <Route path="/about" component={About} />
   </div>
  </HashRouter>
 );
}
```

As we will be serving the application from the /my-react-app subdirectory on our GitHub Pages URL, we will want to add the property basename to our <HashRouter>. This will act as our base URL for all our routes.

Next we will add some views for our routes. Add the following functions to App.js just before we export the component.

```
// App.js
...
const Home = () => <div><h2>Home</h2></div>
const About = () => <div><h2>About</h2></div>
export default App;
```

Now our App.js file should look like the following;

```
import React, { Component } from 'react';
import { HashRouter, Route, Link } from "react-router-dom";

class App extends Component {
  render() {
    return (
      <HashRouter basename="/">
        <div>
          <ul>
            <li><Link to="/">Home</Link></li>
            <li><Link to="/about">About</Link></li>
          </ul>

          <hr />

          <Route exact path="/" component={Home} />
          <Route path="/about" component={About} />
        </div>
      </HashRouter>
    );
  }
}
```

```
const Home = () => <div><h2>Home</h2></div>

const About = () => <div><h2>About</h2></div>

export default App;
```

That's everything we need for a working React application with Routing. Serve the application locally via npm start, and confirm routing is working as expected.

Adding GitHub Pages and deploy the application

For this, we will need to install gh-pages package.

```
$: npm install gh-pages
```

Before we build our application for deployment we need to add a homepage property to our package.json file. This sets the base URL for all our routes and also lets GitHub Pages know where to deploy. We also need to update our scripts property so that we can run a deploy from the command line.

```
// package.json

{

  "name": "my-react-app",

  "homepage": "https://8eni.github.com/my-react-app"

   ....

  "scripts": {

     ...

    "predeploy": "npm run build",

    "deploy": "gh-pages -d build"

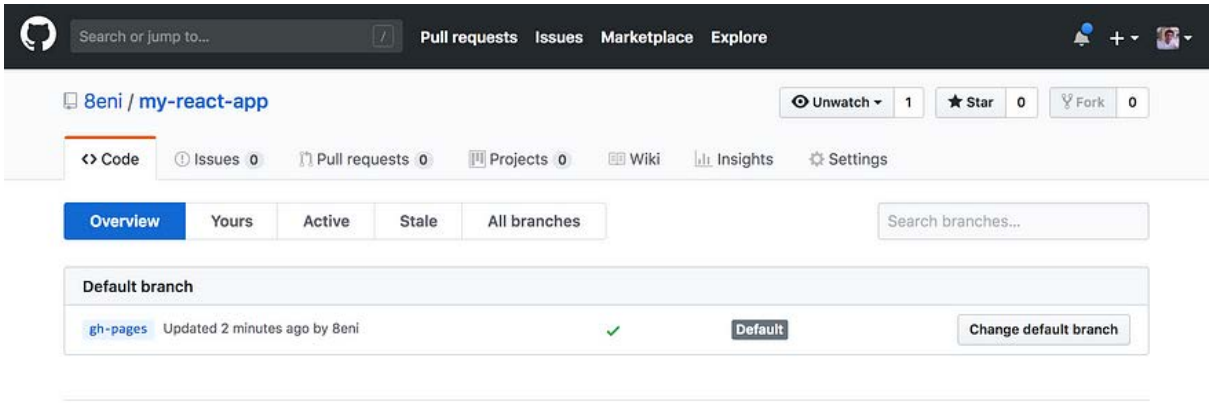
     ...

}
```

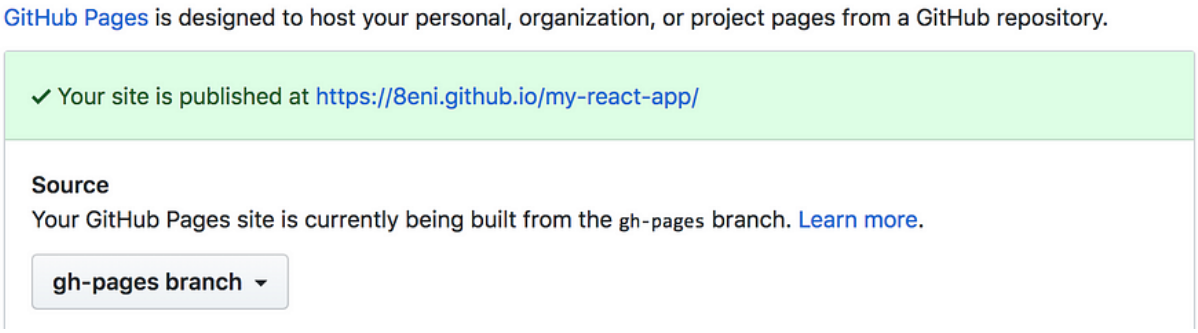
The predeploy script initiates after we run our deploy script, this bundles our application for deployment. Let's deploy our application to GitHub pages;

```
$: npm run deploy
```

If our build and deployment were successful you should see Published in the last line of your terminal, and if you go to your repository on GitHub and you should see a new branch created called gh-pages.

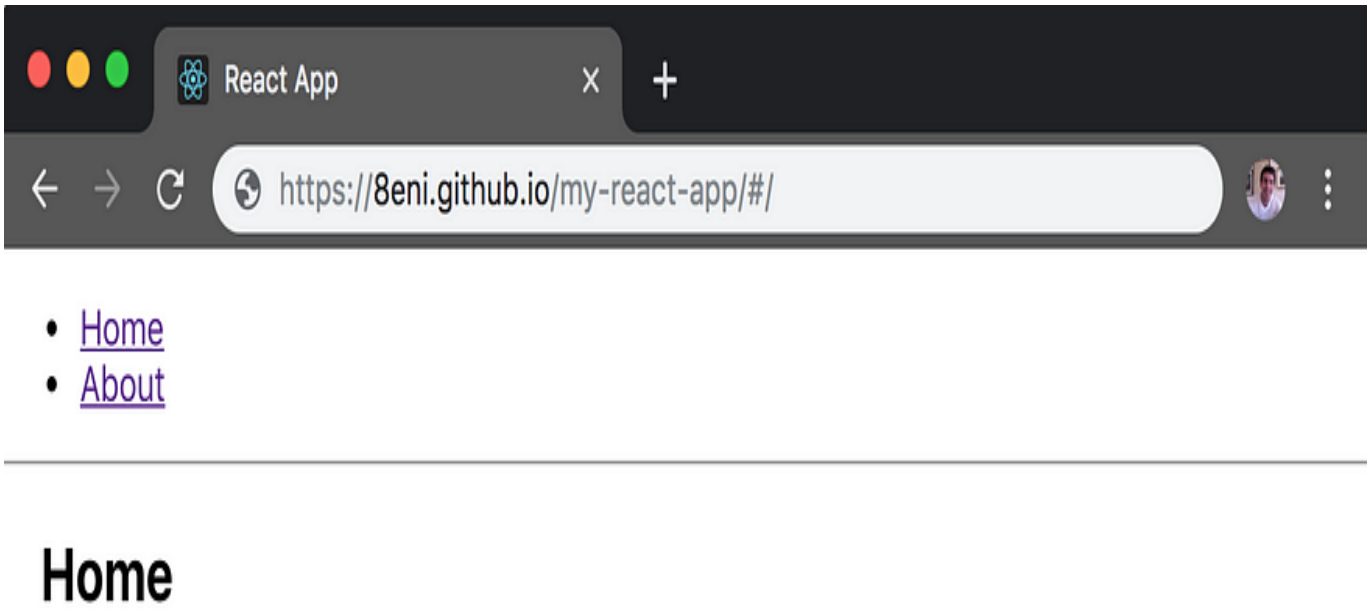


In the settings tabs of your repository you should also see confirmation of deployment to your new URL



GitHub Settings tab — Confirmation our application has deployed successfully

Navigating to our URL we should see our application in action https://8eni.github.io/my-react-app.



7. Conclusion:

We have successfully deployed a create-react-app with routing to a live URL on GitHub Pages. By utilising react-router-dom's HashRouter we don't have to worry about getting a 404 error on page refresh. As we are hosting the application in a subdirectory this means we can deploy multiple applications in different subdirectories on our GitHub Pages URL.

8. Viva Questions:

- What is React.js.
- Define what are Git Repository.
- What are the main advantages of using HashRouter?
- Explain structure of your React.js project.

9. References:

- Ralph Moseley, M.T. Savaliya, "Developing Web Applications", Wilely India, Second Edition, ISBN: 978-81-265-3867-6
- https://legacy.reactjs.org/docs/getting-started.html
- https://www.geeksforgeeks.org/react-tutorial/
- https://www.w3schools.com/REACT/react_intro.asp

Advanced Web Technology

Lab

Experiment No.: 10

Case Study - Integration of SSL Certificate in Web Application

Experiment No. 10

1. **Aim:** Case Study - Integration of SSL Certificate in Web Application

2. **Objectives:** From this experiment, the student will be able to

   Implement SSL Certificate to keep sensitive information sent across the Internet encrypted so that only the intended recipient can access it.

3. **Outcomes:** The learner will be able to
   Authenticate Certificate to a website's identity and prevent attackers from creating fake versions of the site.

4. **Software Required:** Web Browser.

5. **Theory:**

   SSL (Secure Sockets Layer) is a layer of security that can be applied to websites on the internet. It is very useful for sites that require visitors to store personal information. This can include email addresses, names, phone numbers, and credit card information. Using this layer of protection encrypts all data that is sent from a site visitor's browser to the server that hosts the website. While most web development companies in Toronto and across Canada implement SSL as a part of their services, it's important to know why they do it. If SSL is an option when designing a site, make sure you opt for it. It adds a layer of trust between the customer and your company when they see that their information is protected. It also helps that Google's HTTPS Everywhere campaign is in full-swing: the search engine applies a massive bonus in terms of trust rankings and search engine positioning to websites that have an SSL certificate. You will notice the certificate in most browsers: look for the little green padlock symbol next to the website's URL and the "https://" tag in the address bar.

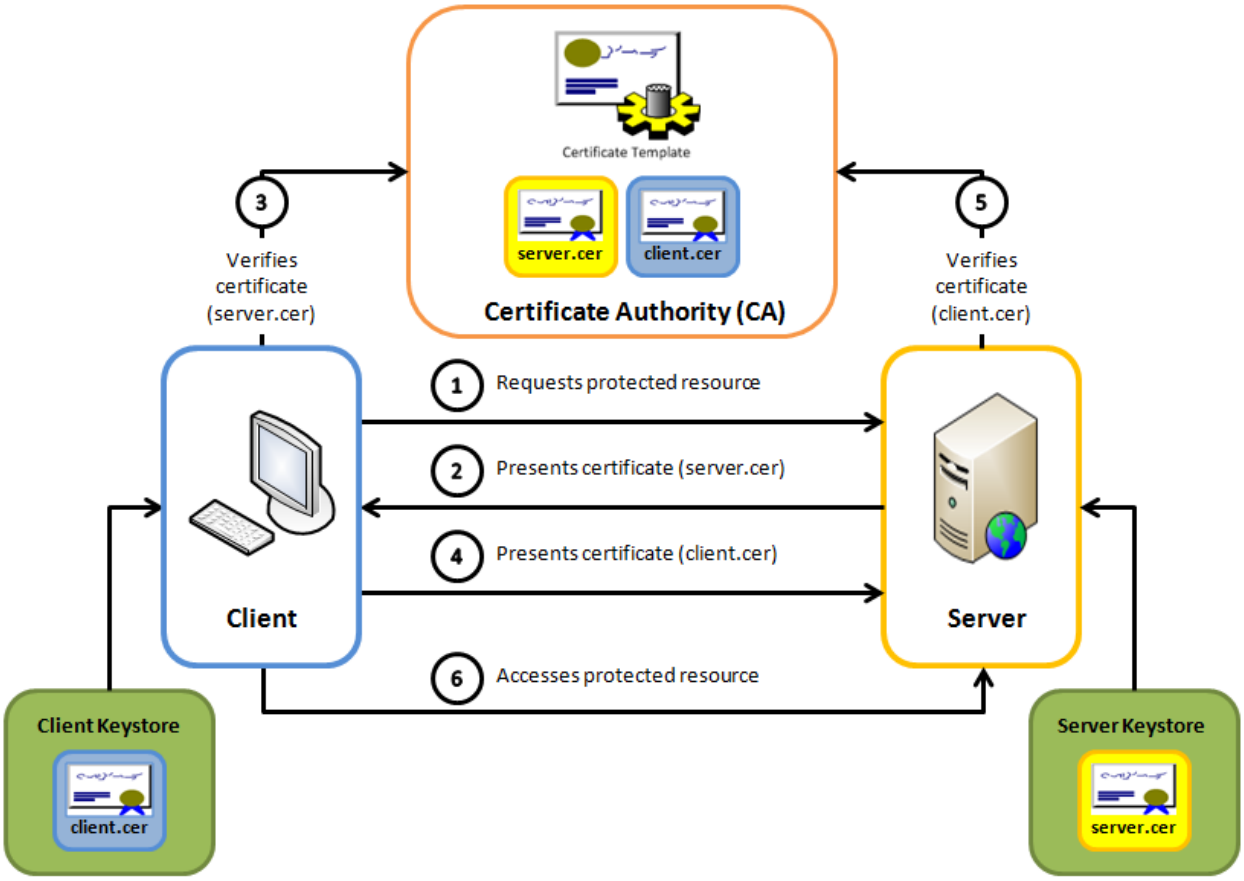
Scenario

A company named Techify Solutions is launching an e-commerce web application. Since the application will handle sensitive customer information, including payment details, they must secure the communication between the users and the server. The best approach is to integrate an SSL certificate and enable HTTPS on their web application.

Steps Involved in SSL Integration

Choosing the Type of SSL Certificate

Single-Domain SSL: Protects a single domain, e.g., www.techifysolutions.com.
Wildcard SSL: Protects a domain and all its subdomains, e.g., *.techifysolutions.com.
Multi-Domain SSL: Protects multiple domains, e.g., www.techifysolutions.com, shop.techifysolutions.com. Techify Solutions decides to use a Wildcard SSL as they have multiple subdomains, including the e-commerce platform and blog.



Mutual SSL authentication / Certificate based mutual authentication

Purchasing the SSL Certificate

SSL certificates can be obtained from trusted Certificate Authorities (CAs) like Comodo, DigiCert, or Let's Encrypt. In this case, Techify Solutions chooses Let's Encrypt for its free, automated certificate service.

Generating a Certificate Signing Request (CSR)

The CSR contains information about the server, including the domain name and the organization's details.

The steps to generate a CSR using OpenSSL:

openssl req -new -newkey rsa:2048 -nodes -keyout techify.key -out techify.csr

This command creates a private key (techify.key) and a CSR file (techify.csr). Submitting the CSR to the Certificate Authority

Techify Solutions submits the CSR to Let's Encrypt for validation. The CA verifies the ownership of the domain before issuing the SSL certificate.

Installing the SSL Certificate on the Server

Once the SSL certificate is issued, it needs to be installed on the server. The web server used in this case is Apache.

The following steps were followed:

The SSL certificate and the CA's intermediate certificates were placed in the server.
The Apache configuration was updated to include the SSL certificate:

```
SSLCertificateFile /etc/ssl/certs/techify.crt
SSLCertificateKeyFile /etc/ssl/private/techify.key
SSLCertificateChainFile /etc/ssl/certs/ca_bundle.crt
```

Apache was restarted to apply the changes:

```
sudo systemctl restart apache2
```

Enabling HTTPS

The web application is configured to force HTTPS by redirecting HTTP traffic to HTTPS. In Apache, this is done by adding the following to the .htaccess file:

```
RewriteEngine On
RewriteCond %{HTTPS} !=on
RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

Testing the SSL Certificate

To verify the SSL installation, tools like SSL Labs' SSL Test can be used to check the security level of the certificate. The test confirmed that the SSL certificate was properly installed and configured, giving an A+ rating.

Maintaining the SSL Certificate

Since Let's Encrypt certificates are valid for 90 days, Techify Solutions set up an automatic renewal process using certbot, a tool provided by Let's Encrypt:

```
sudo certbot renew --dry-run
```

Results

After the successful integration of SSL, the web application runs over HTTPS, ensuring encrypted communication between users and the server. Techify Solutions gained the trust of their users by displaying the secure padlock symbol in the browser's address bar. Performance was improved by enabling HTTP/2, which is supported over HTTPS, offering faster page load times.

Conclusion

Integrating SSL certificates into a web application is a critical step for ensuring secure communication between users and servers. This case study highlights the technical steps and challenges encountered during the SSL integration process. With SSL in place, Techify Solutions successfully secured their e-commerce platform, enhancing customer trust and data security.

6. Viva Questions:

- What is an SSL certificate, and why is it important for a web application?
- What are the different types of SSL certificates, and how do you choose one?
- What is a Certificate Authority (CA), and what role does it play in SSL?
- Can an SSL certificate be transferred to a different server?

7. References:

- Ralph Moseley, M.T. Savaliya, “Developing Web Applications”, Wilely India, Second Edition, ISBN: 978-81-265-3867-6.
- "The Importance of SSL for SEO" - Search Engine Journal, https://www.searchenginejournal.com/ssl-https-seo/
- OpenSSL Documentation: https://www.openssl.org/docs/
- Apache SSL Configuration: https://httpd.apache.org/docs/2.4/ssl/ssl_howto.html
- Let’s Encrypt Documentation - https://letsencrypt.org/docs/
- Comodo SSL Certificates Overview - https://comodosslstore.com